

Frontend Build Plan — Voice-Native App (Rime Hackathon)

This file is instructions for an AI coding agent (or you) to build the **frontend only**. The Python backend (speech recognition, LLM orchestration, Rime TTS calls, state, tools) is a **separate service** built by someone else. The frontend never talks to Rime, an LLM, or a database directly — it only talks to the Python backend's HTTP/WebSocket API.

1. What we're building

A web app with:

Google sign-in (only auth method needed for the hackathon).

Dashboard — landing/home after login.

Text to Speech page — type text, pick a voice, hit generate, play/download audio.

Live Chat page — real-time voice conversation (mic in, Rime audio out), styled loosely like the Gemini homepage (centered input, clean full-height chat canvas) but strictly **black/white/gray, no color**, with a working dark mode toggle.

Chat History — list of past Live Chat sessions with their transcripts, in the sidebar.

The frontend's job in the hackathon rubric: make the "normal interaction" and the "stress case" (interruption, latency, pronunciation — whatever hard problem is chosen) **visible**. That means: a visible "speech provider: Rime" indicator, a visible latency/interrupt state, and a clean transcript so judges can see what happened, not just hear it.

2. Tech stack

Next.js 15/16 (App Router), TypeScript, Tailwind CSS v4.

shadcn/ui for primitives (button, dialog, sheet, tabs, skeleton, scroll-area) — themed to grayscale only, do not use its default colored variants.

Auth: Better Auth (Google provider only) or NextAuth — pick one, do not build custom. Better Auth is preferred if you want the session to live in the same app cleanly.

Data fetching: a single typed API client (`src/lib/api-client.ts`) — no raw `fetch()` calls scattered in components.

Realtime: native `WebSocket` (or `EventSource`/SSE if the backend prefers) for Live Chat streaming — audio chunks and transcript events come over this channel.

Audio: Web Audio API / `<audio>` + `MediaRecorder` for mic capture, a small queue player for streamed TTS chunks so playback doesn't stutter.

State: React state + Context for session/auth; a lightweight store (Zustand) for the live chat session (transcript, connection state, current speaker, interrupt flag) if prop-drilling gets painful. Do not add Redux.

No colored design tokens. Only grayscale (`--background`, `--foreground`, `--muted`, `--border` etc. as varying shades of black/white/gray) plus system-driven dark mode.

3. Why we are adapting `hackathon-struct`, not copying it

That repo assumes the Next.js app **is** the backend (API routes + Drizzle + Neon all in one Next.js deploy, no separate service). We have a real separate Python backend instead, so:

hackathon-struct assumes	We actually do
<code>src/app/api/*/route.ts</code> contains real logic eventually	Next.js API routes are not used for business logic — the Python service owns it
Same-origin API, no CORS	Cross-origin calls to the Python backend → CORS must be configured on the Python side
Drizzle/Neon inside the Next.js repo	No DB code in the frontend repo at all — backend owns persistence
<code>defineRoute(contract)</code> serves mocks locally	We keep the contract + mock idea (typed schema + a <code>mock()</code> fallback) but the typed client calls an external base URL , and mock mode is a client-side toggle, not a local route

What we **keep** from it, because it's genuinely good for parallel frontend/backend work:

Contract-first development: define the shape of every API call (request/response Zod schema) before the backend exists, so the frontend can build against a mock and swap to the real thing with zero component changes.

One typed client, never raw fetch in components.

A visible mock/live badge so nobody accidentally demos fake data as real.

Deploy on day one with a hello-world page.

If you want, you can literally start from the `hackathon-struct` repo and delete `src/db`, `drizzle.config.ts`, the Drizzle-related scripts, and the DB-backed parts of `src/contracts/_kit.ts` / `defineRoute`, keeping only the contract typing + mock pattern and the shadcn/Tailwind setup. Otherwise, scaffold fresh with the structure below — it's less to strip out.

4. Repo structure



```
frontend/
src/
  app/
    (auth)/
      sign-in/page.tsx      # Google sign-in button, minimal
    (dashboard)/
      layout.tsx            # sidebar + topbar shell, session-gated
      dashboard/page.tsx    # landing after login
      text-to-speech/page.tsx
      live-chat/page.tsx
      live-chat/[sessionId]/page.tsx # replay a past session's transcript
  dev/
    api/page.tsx           # lists every contract + mock/live status (from hackathon-struct's /dev/api idea)
    layout.tsx             # root layout, ThemeProvider
    globals.css            # grayscale CSS variables + dark mode
  components/
    ui/                    # shadcn primitives (grayscale themed)
  shell/
    Sidebar.tsx
    Topbar.tsx
    ThemeToggle.tsx
  chat/
    ChatCanvas.tsx         # the Gemini-homepage-like centered input + message stream
    MessageBubble.tsx
    VoiceInputButton.tsx   # mic on/off, waveform indicator
    ProviderBadge.tsx     # "Speech: Rime" indicator, shows fallback if active
    LatencyBadge.tsx      # optional: shows ms since user stopped talking
  tts/
    VoicePicker.tsx
    TtsForm.tsx
    AudioPlayer.tsx
  history/
    HistoryList.tsx        # sidebar list of past sessions
    TranscriptView.tsx
  common/
    MockBadge.tsx
    EmptyState.tsx
    ErrorState.tsx
  contracts/
    _kit.ts                # schema + mock helper types (client-side only now)
    tts.ts                 # POST /tts request/response schema + mock()
    chat.ts                # chat session, message, transcript schemas + mock()
    history.ts             # list/get session schemas + mock()
    index.ts               # append-only registry of the above
```

```
lib/
  api-client.ts      # typed client, talks to NEXT_PUBLIC_API_BASE_URL
  ws-client.ts       # WebSocket wrapper for live chat streaming
  auth.ts / auth-client.ts  # Better Auth config (Google only)
audio/
  recorder.ts        # mic capture helper
  player-queue.ts    # streamed audio chunk playback queue
hooks/
  useLiveChatSession.ts  # connection state, transcript, send/interrupt actions
  useTheme.ts
public/
.env.example
package.json
tsconfig.json
```

5. Design language (no colors, monochrome only)

Palette: pure black, pure white, and 4–6 steps of gray in between. No blue links, no green "success" states, no red "error" states — use weight, borders, icons, and underline/strike instead of color to communicate status.

Typography does the heavy lifting: one clean sans (e.g. Inter or Geist), strong size/weight contrast between headings and body, generous whitespace — similar spirit to the Gemini homepage (a big centered prompt input, minimal chrome) but distinctly ours: sharper corners or a different accent shape, different empty-state copy, our own sidebar-first layout instead of Gemini's centered-everything layout. Dark mode: a real toggle (not just OS-detected), persisted in `localStorage` (client-only; do not use it for auth/session data — see artifact/browser-storage rules if any part of this ever becomes an Artifact, not relevant here since this is a real deployed app). Implement with a `ThemeProvider` flipping a `class="dark"` on `<html>` and CSS variables for the grayscale tokens — do not hardcode `bg-white`/`bg-black` all over components.

Motion: subtle only — fade/slide for messages arriving, a simple pulsing dot or waveform for "listening"/"speaking" states instead of color-coded status pills.

Layout shell

Left sidebar (collapsible on mobile into a sheet): logo/name, then nav items `Dashboard`, `Text to Speech`, `Live Chat`, then a divider, then `Chat History` as a scrollable list of past sessions (title = first user line or timestamp, click to open transcript). Bottom of sidebar: user avatar/email (from Google) + theme toggle + sign out.

Topbar (optional, can fold into page headers): current page title, and on Live Chat specifically, the `ProviderBadge` ("Speech: Rime" / "Fallback: X") so it's demo-visible.

Dashboard page: short welcome, quick links/cards into Text to Speech and Live Chat, maybe a small usage/status readout if the backend exposes one. Keep it simple — this is not the star of the demo.

Text to Speech page: textarea, voice picker (fed by backend's voice catalog endpoint, not hardcoded — the PS explicitly warns against a stale hardcoded speaker list), generate button, resulting audio player

with waveform + download.

Live Chat page: centered layout on first load (empty state, big input/mic button, Gemini-homepage-like), transitioning to a top-anchored transcript + bottom input bar once a session starts. Must show: connection/provider state, an interrupt-in-progress indicator, and (if the chosen hard problem is latency) a visible timer/badge.

6. Auth flow

`/sign-in` — single "Continue with Google" button, nothing else.

On success, redirect to `/dashboard`.

All `(dashboard)/*` routes are session-gated in `layout.tsx` via `await auth.api.getSession(...)` (server-side check) — redirect to `/sign-in` if absent.

Store no PII beyond what the auth provider gives you (name, email, avatar). The Python backend needs to know *who* is chatting (to fetch/save their history) — pass the session's user id or a signed token in a header on every API/WebSocket call; do not pass the whole session object.

7. Talking to the Python backend

Base URL from `NEXT_PUBLIC_API_BASE_URL` env var (e.g. `http://localhost:8000` in dev, the deployed backend URL in prod). All calls in `api-client.ts` prefix with this — never relative `/api/...` paths, since there's no local API route serving real data.

Every feature gets one file in `src/contracts/`: a Zod request schema, a Zod response schema, and a `mock()` returning realistic fake data — so pages can be built and demoed before the backend endpoint exists. Client supports `?__mock=1`-style local override via a simple `USE MOCKS` flag read from env or a dev-only UI toggle, not a server-side switch (there's no local server to switch).

Render a small `<MockBadge>` on Text to Speech / Live Chat / History whenever data came from a mock, so nobody demos fake output as real — mirrors the source-driven badge idea from `hackathon-struct` but driven by the client, not a `source` field from a local route (the real backend should still be asked to include a `source: "mock"|"live"` field in its responses if possible, so the badge reflects backend truth, not just a local flag).

CORS: since frontend and backend are different origins, the backend must send `Access-Control-Allow-Origin` for the deployed frontend origin (and `credentials` handling if you pass cookies). Flag this to the backend owner early — it's the #1 integration blocker in split-repo hackathon projects.

WebSocket contract for Live Chat (agree this with the backend owner explicitly):

Client → server events: `start_session`, `audio_chunk` (binary or base64), `user_interrupt`, `end_session`.

Server → client events: `partial_transcript`, `final_transcript`, `tts_audio_chunk`, `provider_status` (which speech provider is active — Rime vs fallback), `state_sync` (so the UI can reconcile what was actually said if a message was interrupted mid-way — render only the portion of Rime's speech that was actually delivered, per the PS's interruption-and-recovery requirement).

The frontend must handle: reconnect on drop, buffering partial audio chunks into a smooth playback queue, and immediately stopping playback + visually marking a message "interrupted" when

`user_interrupt` fires — this is core to the demo's stress case.

8. Chat History

List endpoint: paginated list of past sessions (id, title/preview, timestamp, duration).

Detail endpoint: full transcript (speaker-tagged turns) + optionally links to stored audio clips if the backend keeps them.

Sidebar renders the list live (not mocked once backend is ready); clicking opens a read-only

`TranscriptView` (same message-bubble component as live chat, just non-interactive).

This view is also useful as demo evidence for the hackathon's "acceptance test" — before/after transcripts for pronunciation fixes, or timestamps showing response latency, can be surfaced here instead of only in a README.

9. Environment variables (`.env.example`)

```
□  
  
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000  
NEXT_PUBLIC_WS_BASE_URL=ws://localhost:8000  
  
# Google auth (Better Auth or NextAuth config)  
GOOGLE_CLIENT_ID=  
GOOGLE_CLIENT_SECRET=  
BETTER_AUTH_SECRET=  
BETTER_AUTH_URL=http://localhost:3000  
  
# never put Rime keys here — those live only in the Python backend's env
```

Never let a Rime API key, Google client secret, or any credential reach client-side code or be committed — only `NEXT_PUBLIC_*` vars are safe to expose to the browser.

10. Hosting & deployment

Frontend → Vercel. One `git push`, static + server components on Vercel's CDN/edge, same as a normal Next.js app. Set `NEXT_PUBLIC_API_BASE_URL`/`NEXT_PUBLIC_WS_BASE_URL` to the deployed backend's URL in Vercel's project env settings (and the Google OAuth authorized redirect URI must include the Vercel production URL).

Backend (Python) → Render / Railway / Fly.io (any always-on host, since it likely needs a persistent WebSocket connection and can't scale-to-zero like a serverless function). This is the backend owner's call, but the frontend needs to know: (a) the deployed base URL, (b) that it must stay on a host that supports long-lived WebSocket connections, (c) that it must set CORS to allow the Vercel domain.

Deploy both on day one with a hello-world page/endpoint each, confirm they can reach each other (a simple `/health` check called from the frontend dashboard is a good early integration test), then iterate.

Do not wait until the end to wire the two together.

Local dev: frontend on `localhost:3000`, backend on `localhost:8000`, `.env.local` points at localhost; no CORS issues locally if both allow `localhost:3000`.

11. Build order (milestones)

Scaffold Next.js + Tailwind + shadcn, grayscale theme tokens, dark mode toggle. Deploy hello-world to Vercel immediately.

Google sign-in wired end-to-end (Better Auth), session-gated `(dashboard)` layout, sidebar + topbar shell with the four nav sections (Dashboard, Text to Speech, Live Chat, Chat History).

Contracts + mocks for `tts`, `chat`, `history`; typed `api-client.ts`; Dashboard and Text to Speech pages built fully against mocks, with `MockBadge` visible.

Live Chat page UI against a mocked/faked WebSocket (simulate transcript + fake audio chunk timing) so the interaction flow (mic button, waveform, transcript stream, interrupt) is fully clickable before the real backend exists.

Swap Text to Speech and Live Chat over to the real Python backend endpoints/WebSocket as they come online — pages should need zero structural changes, only the client base URL and removing mock flags.

Chat History wired to real backend; add the provider/latency badges needed for whichever hard voice problem this project targets, since judges explicitly need to see that evidence in the UI or demo.

Polish: empty states, error states, loading skeletons, mobile sidebar-as-sheet, final pass on "no color anywhere" and dark mode consistency.

12. Open questions to settle with the backend owner before coding

Exact WebSocket message schema (event names/payloads) for Live Chat — agree on this in a shared doc before either side builds, same spirit as "contract first."

Does history come with stored audio, or transcript only?

Where does the `source: mock|live` / `provider: rime|fallback` truth come from — backend response field (preferred) or frontend-only flag?

Auth: does the Python backend need to independently verify the Google session (e.g. via a shared JWT/secret), or does it just trust a user id passed from the frontend for the hackathon's time scale?

Decide the simplest thing that works for a demo.